SMART WATER DISTRIBUTION MANAGEMENT SYSTEM

Review 2 Technical Report — Java Swing Implementation

Department of Computer Science and Engineering | Review 2

Prepared from the current Java Swing source and local regression test results. MySQL persistence code has been added using JDBC; live connection testing still requires local MySQL Server, Connector/J and credentials.

1. Abstract

The Smart Water Distribution Management System is a Java desktop simulation for the fictional town of Neer Nagar. It models reservoirs, demand areas, and pipeline connections; checks reachability using breadth-first search (BFS); and demonstrates allocation requests, partial fulfilment, priority-based batch allocation, pipeline faults, repairs, and maintenance tracking.

The upgraded Swing application preserves the earlier console application and uses standard Java/JDK libraries without external Java dependencies. The current implementation is a simulation, not a hydraulic engineering model.

2. Problem Statement

Water distribution can be disrupted when pipelines fail or available reservoir supply is insufficient. A useful planning tool should show which demand areas remain connected, prevent allocations beyond available supply, prioritize essential institutions, and retain unmet demand for later retry.

Problem scope: model a small network of two reservoirs, ten demand areas, and twelve pipeline links; simulate daily supply and request events; and make connectivity and allocation decisions visible through a desktop dashboard.

3. Objectives

Represent reservoirs, demand areas and pipelines using object-oriented Java classes.

Determine whether a demand area is reachable from a reservoir through working links.

Allow manual requests with automatic selection of a reachable reservoir with the greatest remaining supply.

Support partial allocation and preserve unmet litres.

Offer priority-based and deterministic ID-order batch allocation.

Demonstrate pipeline failure/repair, supply updates, pending-request retries, allocation history and a FIFO maintenance queue.

Provide a live map and dashboard suitable for demonstration and testing.

4. Scope and Assumptions

The sample contains two reservoirs (R1 and R2), ten demand areas (A1–A10), and twelve pipelines (P1–P12). Demand and supply quantities are expressed in litres for a simulated day.

Connectivity is undirected in the current graph model. BFS confirms graph reachability only. The system does not calculate pressure, hydraulic flow, leakage, pump behaviour or physical pipe capacities.

MySQL persistence has been added through JDBC for reservoir balances, pipeline status, water requests, allocation history, event history and maintenance records. It requires local MySQL Server, the MySQL Connector/J runtime JAR, and a configured db.properties file. If the database is not configured or unavailable, the app continues in in-memory mode.

5. Technologies

Language: Java. GUI: Java Swing. Runtime/compiler used for verification: OpenJDK 21. Data structures include LinkedHashMap/HashMap, ArrayList, HashSet, ArrayDeque/Queue, PriorityQueue interface concepts, and collections sorting. Graph representation: adjacency list. Connectivity/path algorithm: BFS.

No external libraries or non-Java front-end technologies are used in the application.

6. Architecture and Class Responsibilities

SwingMain: constructs the Swing dashboard, manages UI events, requests, allocation history, maintenance tickets, preset events and refreshes.

WaterNetwork: owns network entities, stores adjacency-list graph data, checks reachability, returns BFS paths, chooses a reachable reservoir, and changes pipeline state.

Reservoir: stores capacity and remaining supply, caps allocation at available litres, and supports daily reset and supply adjustments.

DemandArea: abstract base class for demand nodes. ResidentialArea, CriticalInstitution and HighPriorityArea override allocationPriority() and display details.

Pipeline: stores endpoints and working/failed status.

WaterRequest: tracks requested, allocated and unmet litres; prevents cumulative recorded allocation from exceeding the original request.

MaintenanceRequest: tracks a pipeline issue and whether the ticket is resolved.

WaterAllocationPolicy / DefaultWaterAllocationPolicy: orders demand areas by Critical > High > Normal, then natural numeric ID; also supports ID-order batch allocation.

Main: original console program retained as a fallback; the expanded network and GUI features are in SwingMain.

7. Core Algorithms and Data Structures

BFS: starts at a reservoir, uses a queue and visited set, and traverses only edges whose Pipeline object is working. Time complexity for standard adjacency-list BFS is O(V + E), excluding the current helper's repeated pipeline lookup.

Adjacency list: stores each node's neighbours and avoids scanning a dense matrix for every traversal.

Maps: provide identifier-based access to reservoirs and demand areas.

Priority ordering: sort by priority descending and then natural numeric identifier order for deterministic tie-breaking. ID-order mode sorts by identifier.

Queue: maintenance requests are stored in FIFO order. Pending water requests can be retried after supply/connectivity changes.

Partial allocation: actual allocation is limited by remaining reservoir supply. WaterRequest records delivered and unmet quantities.

8. Allocation Workflow

1. The user selects a demand area and enters a positive whole-number request.

2. The model checks reachability from available reservoirs.

3. The application selects a reachable reservoir with the greatest remaining supply; ties are resolved deterministically by reservoir ID.

4. The reservoir allocates no more than its remaining supply.

5. WaterRequest records the delivered quantity and remaining unmet quantity.

6. Dashboard metrics, tables, and history refresh.

7. When supply changes or a pipeline is repaired, incomplete requests can be retried.

9. Fault and Repair Workflow

When a working pipeline is failed, its model status changes and BFS no longer traverses that link. The Swing map redraws the failed link in red with a dashed line and break marker. A maintenance ticket is created. On repair, the ticket is resolved, connectivity is recalculated on the next traversal, and pending requests are retried.

Already delivered water is not reversed when a pipeline fails. The application is a simplified connectivity/allocation simulator.

10. Testing and Validation

Compilation command: javac -encoding UTF-8 -d out src/*.java

Regression command: java -ea -cp out ProjectSelfTest

Verified in the current environment using OpenJDK 21: compilation succeeded and 38 model-level checks passed. Checks cover sample size, BFS and unknown-node handling, supply limits, dry reservoir behaviour, reset/clamping, request partial fulfilment and bounds, invalid zero request, pipeline state transitions, disconnection/repair, and deterministic priority/ID ordering.

The regression suite is not a full automated GUI test. Manually verify the visual map, snapshot wrapping, form validation, table refresh, and all visible controls on the review laptop.

11. Test Matrix

Network setup: expected 2 reservoirs / 10 areas / 12 links — PASS.

Connectivity: initial paths and invalid IDs — PASS.

Supply limits: allocation capped at capacity; no negative balance; dry reservoir returns zero — PASS.

Reset/clamping: daily reset restores capacity; supply is clamped to [0, capacity] — PASS.

Request accounting: partial allocation retains unmet litres; repeated allocations cannot exceed request; zero request rejected — PASS.

Pipeline lifecycle: failure, duplicate failure rejection, disconnected branch, repair, duplicate repair rejection, unknown ID handling — PASS.

Priority: critical demand first and normal demand last; deterministic tie-break — PASS.

GUI click-through: not fully automated; manual verification required.

Database persistence: JDBC/MySQL persistence implemented in source; live connection test pending local setup.

12. Sustainability and SDG Relevance

SDG 6 (Clean Water and Sanitation): the simulation demonstrates prioritization and visibility of water availability.

SDG 9 (Industry, Innovation and Infrastructure): the graph and fault-recovery model illustrate infrastructure connectivity and maintenance.

SDG 11 (Sustainable Cities and Communities): the scenario represents how a town may reason about critical services and service continuity.

No measured real-world water savings or environmental impact are claimed. The current project is a small simulation.

13. Limitations and Risks

Persistence depends on MySQL configuration: when connected, the application restores saved state and history; without a configured database, it runs in memory and the session state resets on exit.

Faculty requires a database. MySQL/JDBC persistence code and schema setup files are included. The live database path has not been verified in this environment because no configured MySQL server or Connector/J JAR was available.

Hydraulic pressure, leakage, pipe capacities, physical flow and pump control are not modelled.

Model regression tests do not prove all GUI interactions work; manual click-through is required.

The original console application is retained but uses the original smaller network. Use SwingMain for the expanded demonstration.

14. Immediate Review Demo Plan

1. Start SwingMain and show the dashboard totals.

2. Open Network Map and identify R1, R2, demand areas and pipeline IDs.

3. Allocate a request to A3 and show the remaining reservoir supply change.

4. Demonstrate a request larger than available supply and explain the unmet quantity.

5. Run priority-based and ID-order batch modes and compare the order.

6. Fail P9; show its red broken marker and A9/A10 reachability change.

7. Repair P9; show the link restore and pending requests retry.

8. Show the maintenance queue and allocation/event history.

9. State the simulation assumptions and explain that JDBC/MySQL persistence is implemented but must be connected to the configured local server.

15. MySQL Persistence

The application now includes a MySqlPersistence JDBC component, a MySQL schema script, an example connection-properties file, a Maven dependency declaration for MySQL Connector/J, and a DatabaseConnectionTest smoke-test class. The first successful run seeds the configured network; subsequent runs restore reservoir balances, pipeline statuses, water requests, allocation history, event history and maintenance tickets. Changes are saved in a database transaction after the dashboard refreshes.

Important verification boundary: source compilation and 38 model-level regression checks passed. A live MySQL integration test could not be completed in this environment because no local MySQL server, configured credentials, or Connector/J JAR was available. On the review laptop, run database_schema.sql in MySQL Workbench, copy db.properties.example to db.properties, set credentials, add the Connector/J JAR to lib, and run run-db-check.bat.

16. Conclusion

The Java Swing implementation provides a visible demonstration of graph reachability, reservoir supply constraints, priority allocation, partial fulfilment, pipeline fault handling, maintenance tracking and JDBC/MySQL persistence hooks. The project compiles and its 38 model regression checks pass under OpenJDK 21. Live JDBC integration must be smoke-tested after installing Connector/J, running database_schema.sql and configuring db.properties. Manual GUI verification remains necessary.

16. Review 2 Rubric Traceability

Rubric category	Evidence in current package	Status / gap
Implementation, problem resolution and coding (40)	Java Swing application, graph/model classes, allocation, fault/repair, 38 checks, JDBC/MySQL persistence code	Core simulation implemented; live MySQL connection test and GUI click-through still required
Testing, validation and performance (20)	ProjectSelfTest.java, 38 model checks, DatabaseConnectionTest.java, reproducible commands	38 model checks pass; live DB smoke test pending local MySQL/Connector/J setup; no full GUI automation
Sustainability, cost, resource efficiency and SDGs (10)	SDG 6/9/11 discussion and model assumptions	Qualitative discussion; no real-world impact measurements
Project report (20)	This report, class diagram, test matrix and limitations	Add institution-specific cover page/references and any prescribed format if required
Presentation and viva (10)	Demo sequence and viva script deliverable	Score depends on delivery and ability to explain code